DATABASE MANAGEMENT SYSTEMS • FINAL PROJECT REVIEW 3

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR LIBRARY CIRCULATION AND DIGITAL RESOURCE MANAGEMENT

Final Phase: Advanced SQL Queries, Analytical Views, ACID Transactions, Dual Front-End (Web & Desktop), Systematic Testing & Deployment

Student: Jai Havish Sannapareddy | Roll No: 25WU0102103 | Program: AI/ML | Section: AI/ML RHINOS

Faculty: Miss Sahiti | Submission Date: 15/10/2026

Technology: MySQL 8.0, Python 3, Flask, Tkinter | Milestone: Final Review 3

Implementation Journey: Progress from Review 2 to Review 3

State at Review 2 (Design Phase)

- **Conceptual ER Model:** Entity-Relationship diagram completed with relationships and cardinalities.
- **Normalized Relational Schema:** 15 tables normalized up to 3NF/BCNF.
- **DDL Scripts Created:** `01_schema.sql` written with primary, foreign keys and check constraints.
- **Limitation:** Database was empty; no circulation queries, views, transactions, or user applications existed yet.

Implemented in Review 3 (Final Phase)

- **Comprehensive Data Population:** `02_sample_data.sql` with realistic books, members, issues, fines, and digital logs.
- **Pre-compiled SQL Views:** 6 operational views created (`03_views.sql`) including window functions.
- **Complex SQL Queries:** 10 benchmark queries (`04_queries.sql`) covering multi-joins, aggregates, and subqueries.
- **ACID Transaction Engine:** Python business layer (`library.py`) with atomic transactions and pessimistic locking.
- **Dual Interface Applications:** Built both a desktop GUI (`app.py`) and a modern web application (`web_app.py`).
- **Systematic Unit Testing:** Automated test suite (`test_library.py`) verifying all business rules.

Database Population & Realism (`02_sample_data.sql`)

Realistic Seed Dataset Overview

- **Diverse Book Catalogue:** Populated with real-world computer science, mathematics, physics, and literature textbooks (e.g. Silberschatz Database Concepts, Cormen Algorithms, Rosen Discrete Mathematics) across diverse publishers (MIT Press, McGraw-Hill, Wiley, Prentice Hall).
- **Multi-Tier Member Base:** Registered Student, Faculty, and Public patrons with valid card numbers, emails, and phone numbers.
- **Varied Circulation States:** Created active loans on shelf copies, on-time returned books, and realistic overdue loans to evaluate penalty calculations.
- **Fine Accrual Records:** Populated both settled and outstanding fine records with exact days overdue and fine assessments.
- **Hold Reservation Queues:** Simulated high-demand titles with patrons waiting in ordered queue positions (`WAITING` and `READY`).
- **Digital Resource Logs:** Recorded patron access sessions on online learning platforms (ACM Digital Library, O'Reilly, MIT OCW).

Advanced SQL Query Implementation (`04_queries.sql`)

Joins & Aggregate Evaluations

- **Multi-Table Relational Joins:** 5-table join combining `book`, `edition`, `publisher`, `book_author`, and `author` with `GROUP_CONCAT` to aggregate multi-author names.
- **Aggregations with GROUP BY & HAVING:** Query Q8 groups issues by member and uses `HAVING COUNT(*) >= 3` to isolate frequent borrowers.
- **Category Volume Aggregates:** Query Q9 groups categories with `LEFT JOIN` and computes distinct titles and total physical copies.

Nested Queries & Subqueries

- **Nested NOT IN Subquery:** Query Q10 isolates dead inventory—books that have never been issued—by nesting a subquery selecting issued copy IDs.
- **Dynamic Search Subquery:** Query Q3 powers catalog search matching titles, author names, categories, or ISBNs simultaneously.
- **Fine Accrual Calculations:** Calculates live fines by combining `DATEDIFF(CURDATE(), i.due_date)` with dynamic member policy rates.

Analytical Database Views (`03_views.sql`)

Pre-Compiled Views for Data Abstraction & Real-Time Reporting

- **v_available_titles:** Real-time list of titles with at least one physical copy on the shelf (`status = 'AVAILABLE'`), with distinct authors concatenated.
- **v_overdue_copies:** Joins `issue`, `copy`, and `member` with `DATEDIFF` to compute exact overdue days and accrued fines per borrower.
- **v_member_history:** Complete historical loan audit per member showing issue date, due date, return date, loan status, and paid fines.
- **v_popular_books:** Aggregates circulation counts by book to highlight high-demand textbooks and reading materials.
- **v_reservation_queue:** Uses the SQL Window Function `ROW_NUMBER() OVER (PARTITION BY edition_id ORDER BY queue_no)` to present live queue positions.
- **v_fine_summary:** Monthly revenue reporting aggregating fines assessed, total collected, and still pending.

Transaction Management & Concurrency Control

ACID Properties Implementation

- **Atomicity:** All multi-table mutations (e.g. issuing a book updates `copy.status`, inserts `issue`, and updates `reservation`) run inside a Python transaction context manager (`tx()`). If any step errors, the entire block rolls back.
- **Consistency:** Foreign keys and CHECK constraints prevent corrupt states; borrow limit and fine lock validations precede any write.
- **Isolation:** Transactions prevent dirty and non-repeatable reads using MySQL InnoDB default REPEATABLE READ isolation.
- **Durability:** Committed transactions persist to disk via MySQL write-ahead redo logs.

Pessimistic Concurrency Locking

- **SELECT ... FOR UPDATE:** Used during issue, return, and reservation operations. Locks the target `copy` and `member` rows to prevent race conditions when two librarians attempt to issue the same book simultaneously.
- **Queue Number Serialization:** Locks the edition reservation rows to guarantee sequential, un-duplicated `queue_no` assignment during concurrent reservation requests.
- **Automatic Rollbacks:** Handles integrity errors and custom `LibraryError` exceptions gracefully, reporting clean feedback to users.

Business Logic & Circulation Engine (`library.py`)

Python Business Layer Architecture

- **Shared Codebase:** `app/library.py` contains all circulation logic, shared identically across the desktop GUI (`app.py`), the web app (`web_app.py`), and the automated test suite (`tests/test_library.py`).
- **Input Validation Engine:** Custom regex sanitization: `check_isbn` verifies strict 13-digit format; `check_email` verifies valid email syntax; `check_phone` enforces 10 digits.
- **₹50 Fine Lock Enforcement:** Queries outstanding unpaid fines for the borrower; blocks book checkouts if debt exceeds ₹50.00.
- **Book Return & Dynamic Fine Calculation:** Computes `(today - due_date)` in days and multiplies by `fine_per_day` for that borrower's tier; inserts a `fine` record only if overdue.
- **FIFO Hold Queue Transition:** When a book is returned, `_free_copy()` checks for WAITING reservations. If found, the copy is reserved as `ON_HOLD` and reservation status becomes `READY`.

Front-End Applications: Desktop GUI & Web Interface

1. Desktop GUI (`app/app.py`)

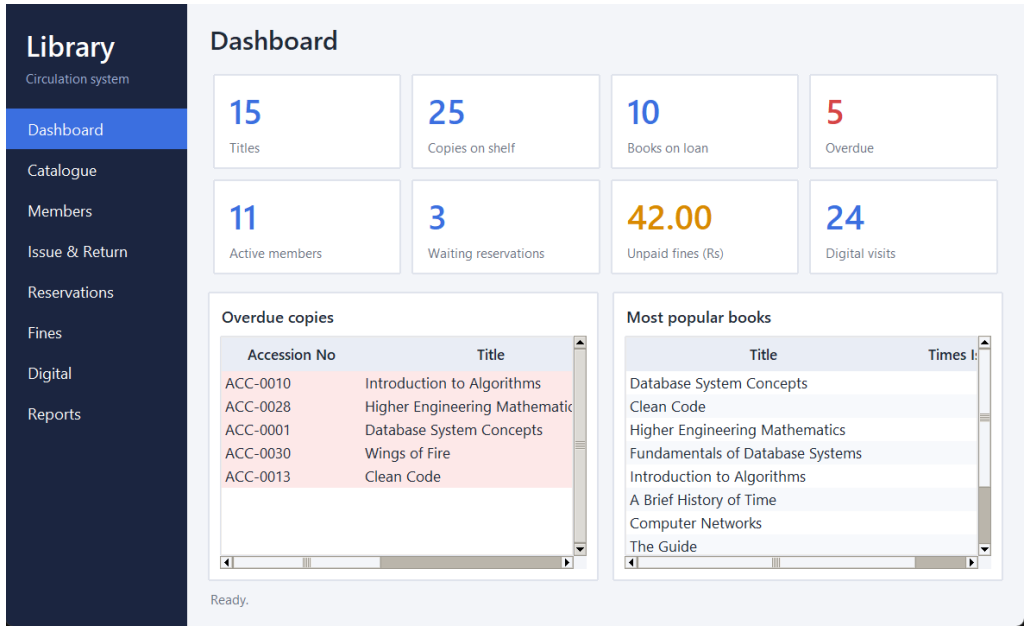
- **Framework:** Python Tkinter with custom `ttk` styling (`clam` engine).
- **Design System:** Deep navy header, card-based layout, Segoe UI typography.
- **Key Screens:** Dashboard overview, Catalogue search & copy adder, Circulation issue/return, Member directory, Reservations queue, Fines ledger, Reports exporter.
- **Best For:** Librarian desktop workstations with local peripheral barcode scanners.

2. Modern Web App (`web_app.py`)

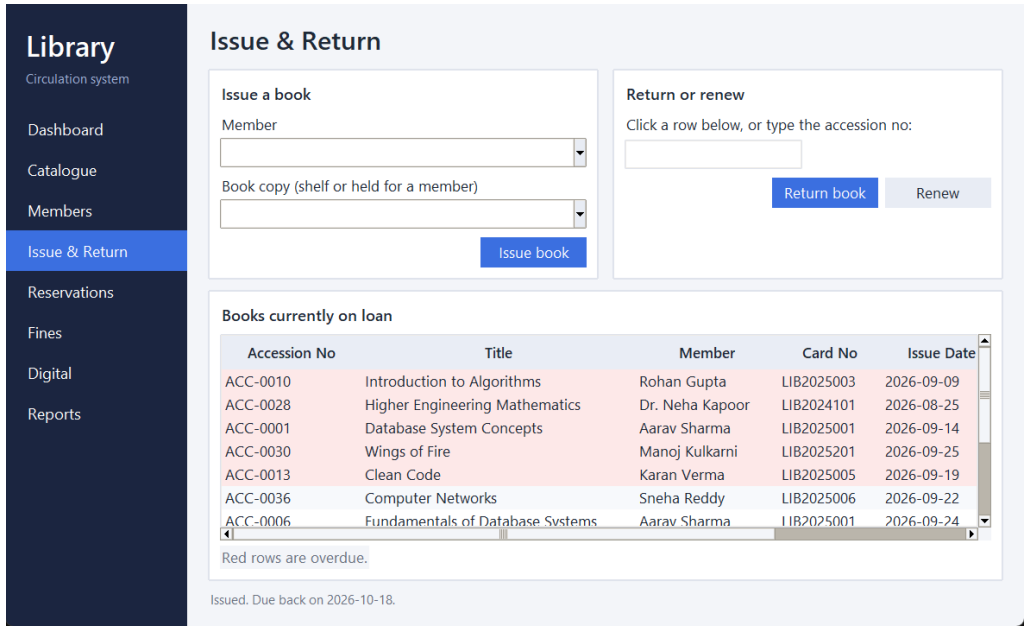
- **Stack:** Python Flask REST API + HTML5 / Tailwind CSS + Chart.js.
- **Design System:** Modern SaaS responsive layout with real-time donut and bar charts.
- **Interactive Features:** Instant live search, animated toast alerts, interactive issue/return dropdowns, member history drawer, 1-click fine payments, and CSV export.
- **Zero-Friction Fallback:** Includes live MySQL connection modal AND zero-setup in-memory fallback demo mode.

Application Demonstration: Core Operational Screens

Executive Dashboard & Overdue Board



Circulation Desk (Book Issue & Return)



Systematic Testing & Rule Verification (`tests/test_library.py`)

Automated Test Suite Methodology & Test Scenarios

- **Isolated Test Harness:** Each test run drops and rebuilds `library_db` from `01_schema.sql`, `02_sample_data.sql`, and `03_views.sql` to guarantee repeatability.
- **Borrow Limit Enforcement Tests:** Verifies that student patrons holding 3 books are strictly blocked upon attempting to check out a 4th book.
- **Fine Lock Verification Tests:** Verifies that an unpaid fine exceeding ₹50.00 blocks borrowing, while a partial payment bringing debt below ₹50 restores borrowing rights.
- **Overdue Math Validation Tests:** Simulates overdue dates and asserts that calculated fines exactly equal `(days_late * fine_per_day)` according to member tier.
- **Reservation Queue Fulfillment Tests:** Tests that returning a reserved book puts the copy into `ON_HOLD`, advances the reservation to `READY`, and prevents other patrons from taking it.
- **Transaction Rollback Tests:** Forces database errors and verifies that partial writes leave zero residual records in `copy` or `issue`.

Technical Achievements & Project Outcomes

1. Data Integrity

- **Normalized 3NF:** Zero data redundancy across titles, authors, editions, and copies.
- **Integrity Constraints:** Strict CHECK and foreign key rules prevent invalid entries at the database engine level.
- **ACID Guarantees:** Atomic transactions protect multi-table updates from corruption.

2. Operational Performance

- **Pre-Compiled Views:** Eliminates repetitive complex join queries for common circulation screens.
- **Indexing Architecture:** Primary and secondary B-Tree indexes speed up lookups on barcodes and ISBNs.
- **Pessimistic Locking:** Prevents concurrency collisions during peak circulation periods.

3. Usability & Deliverables

- **Dual Interface:** Provides both desktop GUI and responsive browser web interface.
- **Automated Tests:** Comprehensive test suite verifying business logic.
- **Complete Documentation:** Formal project reports in Markdown, Word, PDF, and PPTX.

Conclusion & Future Enhancements

Project Summary

- **Complete Lifecycle:** Successfully executed the entire database development lifecycle from requirement analysis, conceptual modeling, normalization, physical implementation, to full-stack application deployment.
- **Objective Achieved:** Replaced uncoordinated registers with an integrated, reliable DBMS enforcing circulation rules, queue order, fine settlement, and digital usage tracking.
- **Ready for Production:** Equipped with dual user interfaces, comprehensive test suite, and database views.

Future Enhancements

- **Hardware Integration:** Direct integration with handheld Barcode / QR / RFID scanners for automated shelf audits.
- **Automated Notifications:** Automated email / SMS webhooks notifying patrons 2 days before due dates and alerting reservation holders when books become READY.
- **Online Student Portal:** Self-service mobile web portal allowing students to view active loans and renew books remotely.